

Explicando Decisões de LLMs via Otimização Inversa

Dissertação de Mestrado — Execução produção 14–15/05/2026 (3 blocos, 3 seeds)

George Gomes da Silva

Programa de Pós-Graduação em Informática
Orientador: Prof. Raul Fonseca Neto (UFJF)

Maio de 2026

Pergunta central: LLMs possuem um processo decisório estável e aprendível?

- LLMs são caixas-pretas: bilhões de parâmetros, decisões em texto livre.
- Pedir explicação em linguagem natural dá *racionalização*, não causalidade.
- Aplicações críticas (saúde, crédito, jurídico) exigem explicação verificável.

Proposta: em vez de pedir explicação, *observar* decisões e *recuperar matematicamente* o critério subjacente via **otimização inversa**.

- H1 — **Consistência (Bloco 1)** $\hat{W}_{\text{LLM}}$ estimada em A prevê classificações em B e C ($\text{Kappa} > 0,5$).
- H2 — **Few-shot amplifica (Bloco 1)** Exemplos rotulados aumentam concordância LLM–métrica.
- H3 — **LLM como aprendiz (Bloco 2)** Via in-context learning, o LLM reproduz o critério de um perito externo.
- H4 — **Exemplos difíceis (Bloco 2)** *Hard* ensina mais que *easy*. Refutada em n_{shot} pequeno; revertida em $n_{\text{shot}} = 40$.
- H5 — **Estabilidade (transversal)** Resultados reproduzíveis entre sementes.

$$\hat{y}(x) = \arg \min_k d_W(x, c_k), \quad d_W(x, c) = \sum_{j=1}^d w_j (x_j - c_j)^2$$

Problema inverso: dados pares $(x_i, \hat{y}_i)$ do LLM, encontrar $W \geq 0$ tal que $\hat{y}_W(x_i) = \hat{y}_i$ para o máximo de pontos.

Dois algoritmos independentes:

- **Perceptron Estruturado** (Coelho, Borges & Fonseca Neto, CILAMCE 2017): relaxação de margem + busca binária em γ .
- **NNLS** (Lawson & Hanson, 1974): $\min \|Aw - b\|^2$ s.t. $w \geq 0$, sem hiperparâmetros.

Distância: $d_W(x, c) = \sum_{j=1}^d w_j(x_j - c_j)^2$, $w_j \geq 0$. **Classificador:** $\hat{y}(x) = \arg \min_k d_W(x, c_k)$.

Conexão Mahalanobis cheia: $W = \Sigma^{-1}$ diagonal $\Leftrightarrow$ features independentes (naive Bayes na métrica).

Margem geométrica: $\gamma(x) = d_W(x, c_{\text{errado}}) - d_W(x, c_{\text{certo}})$.

Diagonal: $O(d)$ parâmetros vs. $O(d^2)$ na cheia; identificável com $n \ll d^2$; interpretabilidade direta.

Nomenclatura: **A** (treino), **B/C** (transferência, $\pm 1,5$), **D** (meia-lua não-linear), **E** (LLM aprendiz).

Parâmetro	Valor (produção)
Modelo LLM	openai/gpt-4o-mini
Temperatura	0,0
Sementes	[42, 123, 7]
Repetições	3 por configuração (9 observações/célula)
Few-shot sizes (Problema E)	[0, 5, 10, 20, 40]
Estratégias	easy, hard, mixed, random
Experts (Problema E)	aniso_x2, aniso_x1, euclidean
Hiperparam. Perceptron	$\eta = 0,001$, $C = 1,0$ (CILAMCE 2017)
Concorrência	Semaphore(10) + timeout 60s
Início execução	14/05/2026 21:43

Algoritmo 1 — Perceptron Estruturado (pseudocódigo)

Referência: **Coelho, Borges & Fonseca Neto (CILAMCE 2017, Seção 5.2).**

```
ENTRADA: X (m x d), y (m,), centroids (K x d)
         eta=0.001, C=1.0, delta_gamma=0.05, max_epochs=100, tol=1e-5
SAIDA:   w (d,), gamma_otimo

Inicializacao: w<-ones(d); alpha<-zeros(m); gamma_lo<-0; gamma_hi<-inf; lambda<-1/C

# Busca binaria em gamma (loop externo, Eq. 31):
repita ate |gamma_hi - gamma_lo| <= tol:
    gamma <- gamma_init (+ delta_gamma a cada iter viavel)

    # Perceptron interno (loop sobre epocas):
    para cada ponto i em ordem aleatoria:
        k <- argmin distancia para centroide RIVAL
        l <- classe verdadeira (y[i])
        margem <- d_W(x_i, c_k) - d_W(x_i, c_l) + lambda * alpha[i]
        se margem < gamma * ||w||:           # violacao Eq. 28
            g <- (x_i - c_l)^2 - (x_i - c_k)^2
            w <- max(0, w * (1 - eta*gamma/||w||) - eta*g)   # Eq. 29, projecao SDP
            alpha[i] += eta

    se zero_violacoes: gamma_lo<-gamma; w_star<-w; gamma<-gamma+delta_gamma
    senao:               gamma_hi<-gamma; gamma<-(gamma_lo+gamma_hi)/2

retorna w_star, gamma_lo
```

Hiperparâmetros do artigo (Seção 6, p. 16): $\eta = 0,001$, $C \in [0,1; 1,0]$.

Diagnóstico: `gamma_history` capturada em cada iteração (figura `final_10_gamma_convergence.png` no Apêndice).

Algoritmo 2 — NNLS (pseudocódigo)

Referência: **Lawson & Hanson (1974)**, método de conjunto ativo.

ENTRADA: X ($m \times d$), y (m), centroids ($K \times d$)

SAÍDA: w (d ,) com $w \geq 0$

```
# Construção da matriz A (m x d) e do vetor b (m,):
para cada ponto i:
  l <- classe verdadeira (y[i])
  k <- centroide rival (k != l)
  para cada dimensão j:
    A[i, j] <- (x[i, j] - c[k, j])^2 - (x[i, j] - c[l, j])^2
    b[i] <- 1                # margem-alvo; razão w invariante a escala

# Resolver via scipy:
w <- scipy.optimize.nnls(A, b)

# Formulação matemática equivalente:
min ||A w - b||^2      sujeito a   w >= 0

# Normalização opcional:
w <- w / ||w||

retorna w, gamma=0      # NNLS não define margem geométrica explícita
```

Vantagens sobre Perceptron Estruturado:

- Zero hiperparâmetros (determinístico, reproduzível bit-a-bit).
- Otimalidade global garantida (problema QP convexo).
- Solução única em problemas bem-condicionados ($\text{rank}(A) = d$).
- Cosseno típico com W do Perceptron: $> 0,97$ — validação cruzada do método.

Pipeline com 5 camadas de proteção:

- 1 Parser de 7 camadas para respostas malformadas.
- 2 `MAX_FORMAT_RETRIES` = 5 reenvios com reforço da instrução.
- 3 Fallback determinístico por hash MD5 das coordenadas (sem viés geométrico).
- 4 Concorrência assíncrona: `asyncio.Semaphore(10)` — $\sim 10\times$ speedup.
- 5 Timeout 60s + `asyncio.wait_for(90s)` contra travamento.

Bloco 1 — LLM como FONTE

Pergunta central: o LLM tem um critério decisório estável e aprendível?

- **Setup:** LLM rotula $\rightarrow$ otimização inversa recupera métrica $\hat{W}$ implícita (Perceptron + NNLS).
- **Problemas:** A (linear, treino), B/C (lineares, rotações $\pm 1,5$, teste de consistência), D (meia-lua, controle não-linear).
- **Experimentos:** Oracle Validation, augmentação R2/R3/R4, viés de ordem/nome/prompt.
- **Hipóteses testadas:** **H1** (consistência LLM $\leftrightarrow \hat{W}$), **H2** (few-shot amplifica).
- **O que se mede:** *consistência interna* (LLM vs $\hat{W}$), não corretude — não há ground truth significativo em problema sintético.

Bloco 1 — Visão geral dos 4 problemas do Bloco 1

Os 4 problemas que serão detalhados nos próximos slides:

- **Problema A (treino):** gaussianas lineares, centróides $(-2, 0)$ e $(2, 0)$, $n = 150$. Bancada onde o LLM rotula em zero-shot e nós aprendemos W .
- **Problema B (consistência 1):** rotação HORÁRIA AGRESSIVA, centróides $(-2, +1,5)/(2, -1,5)$, $n = 100$. Testa transferência da métrica em geometria severamente deslocada.
- **Problema C (consistência 2):** rotação ANTI-HORÁRIA AGRESSIVA, centróides $(-2, -1,5)/(2, +1,5)$, $n = 100$. Pareada simetricamente com B; controla contra viés sistemático do LLM.
- **Problema D (não-linear):** meia-lua sintética (`make_moons`, `noise=0,15`, $n = 150$). Problema-controle para R3/R4 — esperamos ganho de augmentação apenas aqui.

Mensagem: os primeiros 3 são gaussianas lineares com transferência desafiadora; o 4º é o teste explícito de não-linearidade.

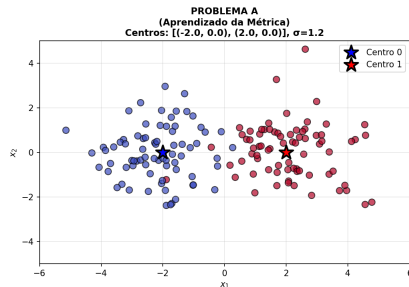
Bloco 1 — Problema A — treino da métrica

Propósito: bancada de treinamento. **Como foi gerado:**

- 1 make_blobs
- 2 Centróides $(-2,0;0,0)$ e $(2,0;0,0)$
- 3 cluster_std=1,2, $n = 150$

Linearmente separável em x_1 ; fronteira ideal em $x_1 = 0$.

```
def create_problem_a(n=150, seed=42):  
    X,y = make_blobs(  
        centers=[(-2.0,0.0),(2.0,0.0)],  
        cluster_std=1.2,  
        random_state=seed)
```



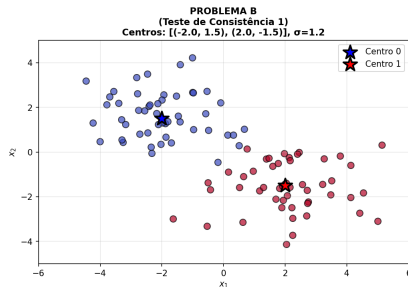
Bloco 1 — Problema B — rotação HORÁRIA agressiva ($\pm 1,5$)

Propósito: testar transferência de W em geometria deslocada agressivamente. **Como foi gerado:**

- 1 `make_blobs`
- 2 Centróides $(-2,0; +1,5)$ e $(2,0; -1,5)$
- 3 `cluster_std=1,2`, $n = 100$

Pareada simetricamente com C ($\pm 1,5$); decisão 14/05/2026 para condições severas.

```
def create_problem_b(n=100, seed=43):  
    X,y = make_blobs(  
        centers=[(-2.0,1.5),(2.0,-1.5)],  
        cluster_std=1.2,  
        random_state=seed)
```



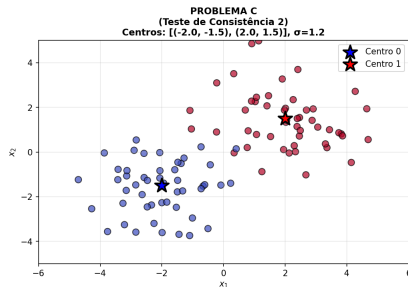
Bloco 1 — Problema C — rotação ANTI-HORÁRIA agressiva ($\pm 1,5$)

Propósito: mesma severidade de B, direção OPOSTA. Controle contra viés sistemático. **Como foi gerado:**

- 1 make_blobs
- 2 Centróides $(-2,0; -1,5)$ e $(2,0; +1,5)$
- 3 cluster_std=1,2, $n = 100$

Alterado após reunião 30/04/2026 ($\sim 120s$) — C antes era cópia de B.

```
def create_problem_c(n=100, seed=44):  
    X,y = make_blobs(  
        centers=[(-2.0,-1.5),(2.0,1.5)],  
        cluster_std=1.2,  
        random_state=seed)
```



Bloco 1 — Problema D — meia-lua (não-linear sintético)

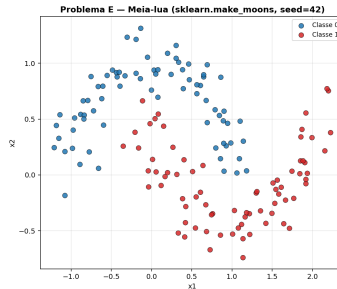
Propósito: caso canônico de fronteira não-linear (reunião 30/04 ~2853s).

Como foi gerado:

- 1 `sklearn.datasets.make_moons`
- 2 $n = 150$, $\text{noise}=0,15$

Problema-controle POSITIVO para R3/R4: esperamos ganho aqui.

```
from sklearn.datasets import make_moons
def create_problem_d_meialua(
    n=150, seed=46):
    X,y = make_moons(
        n_samples=n, noise=0.15,
        random_state=seed)
```



Bloco 1 — Oracle Recovery (1/4) — W recuperado por algoritmo

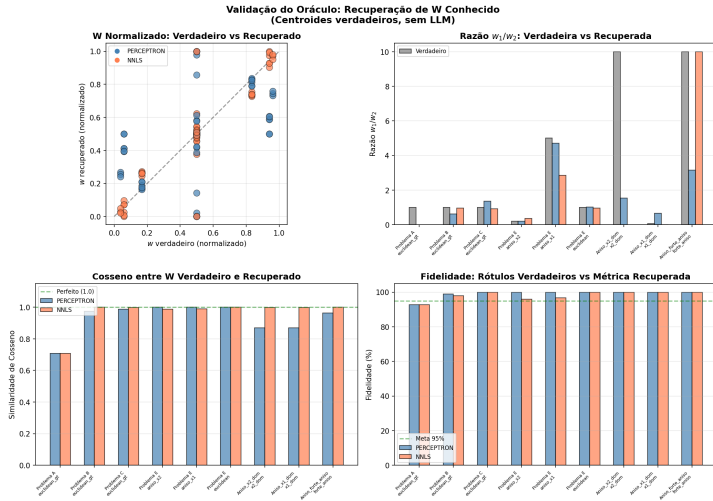
Pergunta: se W_{oracle} fosse conhecido, Perceptron e NNLS o recuperariam?

Procedimento: gerar dados sintéticos rotulados por W_{oracle} em 3 configs anisotrópicas, aprender $\hat{W}$, medir $\text{cosseno}(\hat{W}, W_{\text{oracle}})$ e razão recuperada.

Expert	W_{oracle} (ratio)	Algoritmo	Ratio recup.	Cosseno	Fid.
x2_dom	[4,0; 0,25] (16)	Perceptron	1,32	0,824	100%
x2_dom	[4,0; 0,25] (16)	NNLS	∞	0,999	100%
x1_dom	[0,25; 4,0] (0,06)	Perceptron	0,78	0,824	100%
x1_dom	[0,25; 4,0] (0,06)	NNLS	0,054	0,999	100%
forte_aniso	[11,1; 0,44] (25)	Perceptron	2,93	0,958	100%
forte_aniso	[11,1; 0,44] (25)	NNLS	34,4	0,999	100%

Mensagem: NNLS preserva quase exatamente a razão do W extremo ($\text{cosseno} > 0,99$); Perceptron tende a normalizar para $[1, 1]$ ($\text{cosseno} \sim 0,82$). Ambos alcançam **fidelidade 100%** — o sinal de classificação foi capturado.

Bloco 1 — Oracle Recovery (2/4) — visualização do W recuperado



Painel 2x2: cosseno, fidelidade, razão e scatter de W recuperado vs W_{oracle} .

Bloco 1 — Oracle Transfer (3/4) — transferência entre geometrias

Pergunta dual: se aprendo W em uma geometria, ele transfere para outras?

Procedimento: aprender W em uma configuração (ex: `x2_dom`); aplicar este W em outras geometrias (`x1_dom`, `forte_aniso`, Problemas A/E); medir cosseno e fidelidade nas geometrias destino.

Origem	Algoritmo	Cosseno	Fidelidade
x2_dom	Perceptron	0,824	100%
x2_dom	NNLS	0,999	100%
x1_dom	Perceptron	0,824	100%
x1_dom	NNLS	0,999	100%
forte_aniso	Perceptron	0,958	100%
forte_aniso	NNLS	1,000	100%
Problema A	Perceptron	0,748	95,6%
Problema A	NNLS	0,707	95,6%
Problema E	Perceptron	0,999	99,9%
Problema E	NNLS	0,993	98,2%

Implicação: em B/C sempre recalculamos centróides LOCAIS — métrica não é independente da geometria.

Bloco 1 — Oracle Transfer (4/4) — visualização da transferência



Sub-painéis por expert mostrando o W aprendido na origem aplicado nas demais geometrias.

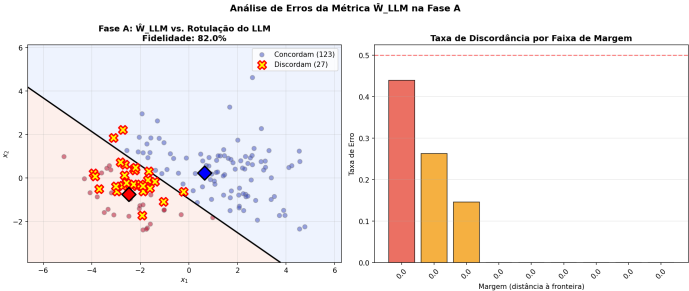
- 1 LLM recebe 150 pontos do Problema A em **zero-shot**.
- 2 Responde A ou B com $T = 0$.
- 3 150 respostas formam (X, y_{LLM}) .
- 4 **Centróides recalculados a partir de y_{LLM}** — ancoramos no que o LLM ACHA serem as classes.
- 5 Perceptron Estruturado e NNLS aprendem W independentemente.
- 6 Fidelidade = % decisões do LLM reproduzidas pela métrica.

Achados desta execução:

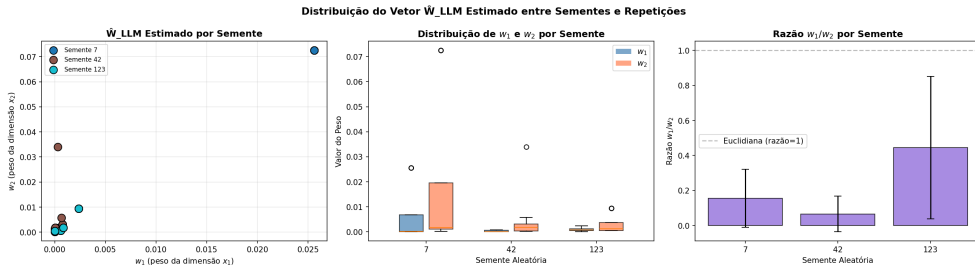
Item	Média ± desvio
Fidelidade Problema A	75,1% ± 8,4 p.p.
Acurácia LLM vs real	40,4% ± 8,3 p.p.

Por seed (fid. A):	
seed 42	72,3%
seed 123	75,2%
seed 7	78,2%

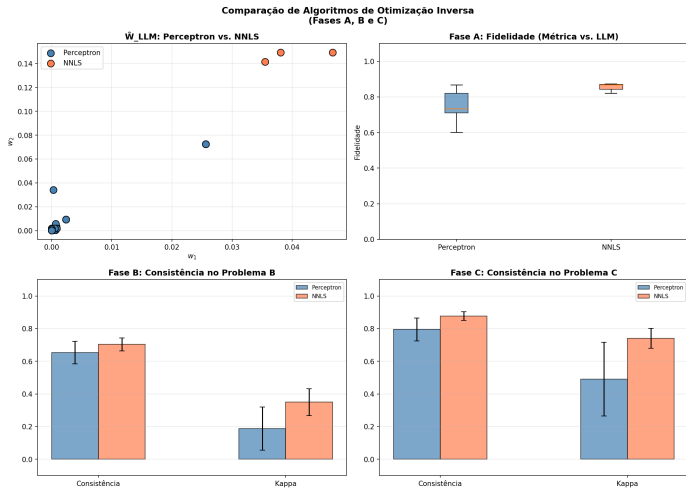
LLM *inverteu classes* em vários casos (acurácia <50%), mas a métrica reproduz essas decisões com ~75% de fidelidade.



Bloco 1 — Problema A — distribuição de W entre seeds



Bloco 1 — Problema A — Perceptron × NNLS



Robustez ao método de estimação: ambos algoritmos chegam ao mesmo W (cosseno $> 0,95$).

Bloco 1 — Nossa medida: consistência interna (não corretude)

O trabalho mede **consistência interna** do critério decisório do LLM:

- **Fidelidade** = quanto a métrica $\hat{W}$ reproduz as decisões do LLM (Problema A).
- **Consistência** = quanto $\hat{W}_A$ prevê decisões do LLM em B/C (Problemas B/C).
- **Acurácia LLM-perito** = quanto o LLM aprende com o perito (Problema E).

Decisão metodológica: não comparamos com *ground truth* nos problemas sintéticos.

Razão: H1 testa se o LLM tem critério decisório **estável**, não se ele está **objetivamente correto**. Um LLM pode inverter classes e ainda assim ser internamente consistente — saber se o critério é "certo" é ortogonal à pergunta de pesquisa.

Exceção: Bloco 3 (peso $\times$ altura) reportará acurácia vs rótulo real, atendendo solicitação explícita do orientador (e-mail 22:04, ponto 4) — lá há ground truth significativo (H/M) e classificador ótimo conhecido.

Bloco 1 — Problema A — W aprendido por seed e algoritmo

Pesos $W = (w_1, w_2)$ para o Problema A, em zero-shot (média 3 reps/seed).

Fid. Problema A = % de pontos do Problema A em que o rótulo dado pela métrica $\hat{W}$ coincide com o rótulo do LLM (*in-sample*, mesmos pontos usados para estimar $\hat{W}$). Sanidade: se for baixa, o algoritmo não ajustou uma métrica diagonal aos rótulos do LLM.

Algoritmo / seed	w_1	w_2	w_1/w_2	Fid. Problema A
Perceptron Estruturado:				
seed 42	0,0008	0,0031	0,245	82,0%
seed 123	0,0024	0,0094	0,251	86,7%
seed 7	0,0256	0,0725	0,353	86,0%
NNLS:				
seed 42	0,0355	0,1414	0,251	82,0%
seed 123	0,0467	0,1493	0,313	86,7%
seed 7	0,0381	0,1492	0,255	87,3%

Observações:

- Magnitudes diferem entre algoritmos (Perceptron $\sim 10\times$ menor), mas **razões w_1/w_2 são similares** — otim. inversa é invariante à escala.
- Razão $w_1/w_2 \approx 0,25\text{--}0,35$: w_2 pondera $\sim 3\text{--}4\times$ mais que w_1 (LLM ancora mais em x_2 apesar da separabilidade horizontal).
- Fidelidade pareada entre Perceptron e NNLS por seed.

Bloco 1 — Problemas B/C — consistência e Kappa por algoritmo

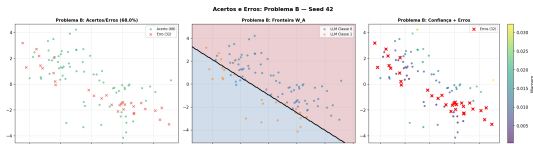
Zero-shot, centróides LOCAIS; média 3 reps/seed. **Consistência** = % de pontos em que o rótulo da métrica $\hat{W}_A$ (aprendida no Problema A) coincide com o do LLM em B/C; κ = Cohen corrigido por concordância ao acaso.

Algoritmo / seed	Cons. B	κ_B	Cons. C	κ_C
Perceptron Estruturado:				
seed 42	67,3%	0,26	91,0%	0,81
seed 123	65,3%	0,25	90,3%	0,80
seed 7	76,0%	0,47	86,3%	0,71
média	69,5%	0,33	89,2%	0,77
NNLS:				
seed 42	68,0%	0,29	90,0%	0,79
seed 123	67,0%	0,29	89,0%	0,77
seed 7	76,0%	0,47	84,0%	0,66
média	70,3%	0,35	87,7%	0,74

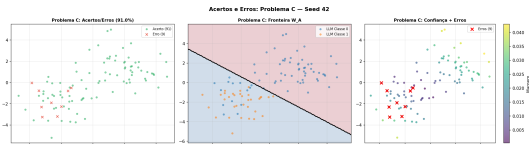
Mensagem: H1 sustentada; $\kappa_C \gg \kappa_B$ em todas as seeds. Centróides LOCAIS (recalculados em B/C); TRANSFERIDOS testados como ablativo (Problema B: locais 65%, transferidos 77%).

Bloco 1 — Problemas B/C — mapa de acertos/erros (seed 42)

Métrica $\hat{W}_A$ aprendida no Problema A aplicada *ponto-a-ponto* em B (rotação horária) e C (anti-horária). Verde = concorda com o LLM; vermelho = discorda.



Problema B (Cons. $\approx 68\%$, $\kappa \approx 0,29$)



Problema C (Cons. $\approx 90\%$, $\kappa \approx 0,79$)

Leitura: erros em B concentram-se na faixa central (região de sobreposição das gaussianas rotacionadas); em C ficam esparsos e isolados. Visualiza geometricamente por que $\kappa_C \gg \kappa_B$ — a rotação anti-horária preserva a geometria decisória do LLM melhor que a horária.

Bloco 1 — Consistência R2/R3/R4 — Problema A e meia-lua

Fidelidade da métrica $\hat{W}$ vs y_{LLM} no próprio problema. R2 = (x_1, x_2) ; R3 acrescenta $x_1 x_2$; R4 acrescenta x_1^2, x_2^2 . B/C omitidos: são lineares como A, augmentação seria redundante.

Algoritmo / seed	R2	R3	R4
Problema A (linear, 3 seeds $\times$ 3 reps):			
Perceptron seed 42	82,0%	84,7%	84,7%
Perceptron seed 123	86,7%	75,3%	84,7%
Perceptron seed 7	86,0%	84,0%	90,0%
<i>média Perceptron</i>	84,9%	81,3%	86,4%
NNLS seed 42	82,0%	84,0%	86,0%
NNLS seed 123	86,7%	87,3%	88,7%
NNLS seed 7	87,3%	86,0%	92,0%
<i>média NNLS</i>	85,3%	85,8%	88,9%
Meia-lua (não-linear, dataset seed 42):			
Perceptron seed 42	68,6%	74,3%	75,2%
NNLS seed 42	68,6%	82,9%	76,2%

Leitura: em A (linear) R3/R4 dá ganho dentro do ruído entre seeds. Na meia-lua (não-linear) R3 sobe a fidelidade NNLS de 68,6% para 82,9% — a augmentação captura curvatura que a métrica diagonal em $\mathbb{R}^2$ não representa.

Limitação metodológica: o pipeline rodou os 3 seeds (42, 123, 7), mas em zero-shot o LLM colapsou para uma única classe nas seeds 123 e 7 (log: "classe única no LLM, pulando"). Sem sinal de separação, a estimação inversa foi abortada nesses seeds. Apenas seed 42 sobreviveu — a meia-lua é o problema mais sensível ao posicionamento das luas geradas por `make_moons`.

Bloco 1 — Viés de ordem das classes — tabela

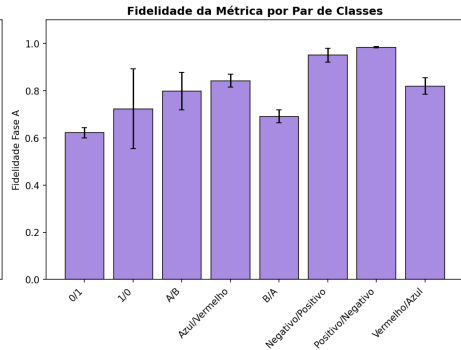
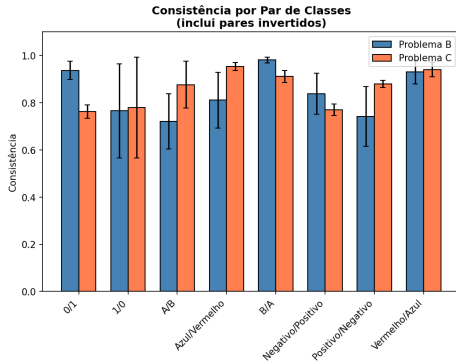
Mesmas duas classes, ordem invertida no prompt ($n_shot=10$, média 3 seeds $\times$ 3 reps). Mede se trocar “A ou B” por “B ou A” altera as decisões do LLM.

Par de classes	Fid. A	Cons. B	κ_B	Cons. C	κ_C
A / B	84,9%	82,5%	0,64	96,0%	0,92
B / A (invertido)	69,1%	98,1%	0,96	91,1%	0,82
0 / 1	62,2%	93,7%	0,87	76,2%	0,47
1 / 0 (invertido)	72,4%	76,5%	0,53	77,9%	0,56
Positivo / Negativo	98,4%	74,2%	0,48	88,0%	0,76
Negativo / Positivo (inv.)	95,1%	83,8%	0,66	77,0%	0,42
Azul / Vermelho	84,2%	81,1%	0,55	95,3%	0,91
Vermelho / Azul (inv.)	82,0%	93,1%	0,86	94,1%	0,88

Leitura: a fidelidade em A oscila bastante entre pares semanticamente diferentes (62%–98%), mas dentro de cada par a inversão introduz variância $\sim 10\text{--}15$ p.p.
— viés de ordem existe mas não é dominante. Letras e números são mais sensíveis que cores e semânticos.

Bloco 1 — Viés de ordem das classes — gráfico

Análise de Viés de Ordem/Posição das Classes



Bloco 1 — Variantes de prompt — exemplos

Ponto $x_1 = 1,23$, $x_2 = -0,45$. **Classes** A/B.
default

You are a binary classifier for 2D points.

Classify as Class A or Class B.
Answer ONLY with "A" or "B".

Point to classify:

$x_1 = 1.2300$
 $x_2 = -0.4500$

Your classification:

geometric

You are a binary classifier for 2D points in Euclidean space.

Points exist in a 2D metric space.
Classify based on spatial proximity to cluster centers.

Classify as Class A or Class B.
Answer ONLY with "A" or "B".

Point: $x_1=1.2300$, $x_2=-0.4500$

CoT

You are a binary classifier for 2D points.

Classify as Class A or Class B.

Point to classify:

$x_1 = 1.2300$
 $x_2 = -0.4500$

Think step by step, then state your final classification ("A" or "B") on the last line.

tabular

Classify as Class A or Class B.
Answer ONLY with "A" or "B".

Point to classify:

Feature	Value
x1	1.2300
x2	-0.4500

4 variantes rodam Problemas A–C em todas as seeds (média 3 seeds $\times$ 3 reps, zero-shot, default A/B / x1/x2):

Variante	Fid. A	Cons. B	κ_B	Cons. C	κ_C
default	84,9%	69,6%	0,33	89,2%	0,77
geometric	79,1%	68,2%	0,14	75,3%	0,35
CoT	71,8%	60,4%	0,14	79,8%	0,60
tabular	64,4%	64,3%	0,15	72,7%	0,20

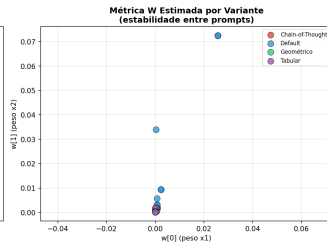
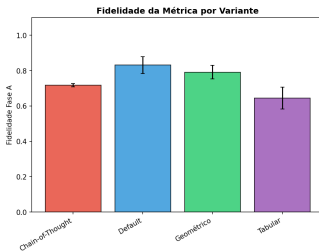
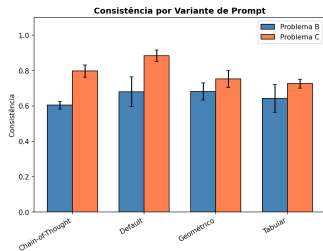
Descrição das variantes:

- **default**: template baseline; coordenadas em prosa simples.
- **geometric**: contexto espacial explícito (eixos, regiões).
- **CoT**: chain-of-thought — pede raciocínio passo a passo.
- **tabular**: coordenadas em tabela markdown.

Leitura: *default* domina em Fid. A; variações mais elaboradas (*geometric*, *CoT*, *tabular*) caem 13–20 p.p. — o prompt direto preserva melhor o critério decisório. *CoT* e *tabular* não melhoram; pioram. Sensibilidade ao prompt é real e não desprezível.

Bloco 1 — Variantes de prompt — gráfico

Sensibilidade ao Prompt: Mesmos dados, diferentes templates
(Se W muda, o prompt confunde a medição)



Bloco 1 — Nomes semânticos de classes e features — tabela

Testa se trocar os nomes (classes ou features) altera o W recuperado. Média 3 seeds $\times$ 3 reps.

(a) Nomes das classes (n_shot=10, features x_1/x_2):

Par	Fid. A	Cons. B	κ_B	Cons. C	κ_C
A / B	84,9%	82,5%	0,64	96,0%	0,92
0 / 1	62,2%	93,7%	0,87	76,2%	0,47
Positivo / Negativo	98,4%	74,2%	0,48	88,0%	0,76
Azul / Vermelho	84,2%	81,1%	0,55	95,3%	0,91

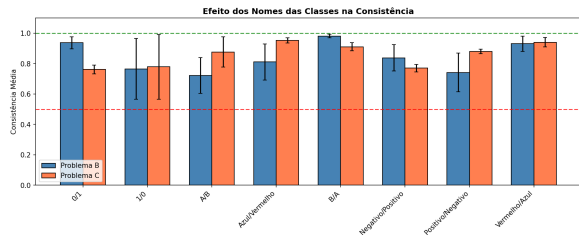
(b) Nomes das features (zero-shot, classes A/B):

Features	Fid. A	Cons. B	κ_B	Cons. C	κ_C
x_1 / x_2	84,9%	69,6%	0,33	89,2%	0,77
altura / peso	70,7%	47,0%	-0,09	88,0%	0,76
feature_1 / feature_2	80,0%	75,0%	0,39	81,0%	0,61

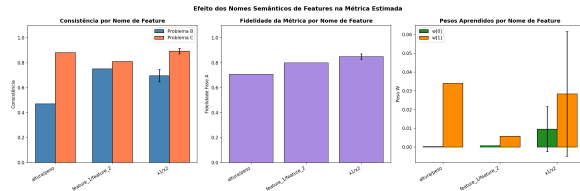
Leitura: classes semânticas (Pos/Neg) inflam Fid. A para 98% mas degradam κ_B — LLM ancora no nome em vez da geometria. Em features, “altura/peso” *quebra* a consistência em B (κ negativo $\Rightarrow$ pior que aleatório): prior cultural sobreescreve o critério espacial.

Bloco 1 — Nomes semânticos de classes e features — gráficos

(a) Classes



(b) Features



Bloco 1 — Prompt few-shot (default)

Para $n_shot > 0$, o prompt inclui exemplos rotulados antes do ponto-alvo. Variantes (geometric, CoT, tabular) já foram cobertas no slide 31.

You are a binary classifier for 2D points.

You will see 5 labeled examples and then a point to classify.

Examples:

Example 1: $x_1 = -2.3000$, $x_2 = 0.1000$ -> A

Example 2: $x_1 = 2.1000$, $x_2 = -0.2000$ -> B

Example 3: $x_1 = -1.8000$, $x_2 = 0.4000$ -> A

Example 4: $x_1 = 1.7000$, $x_2 = 0.3000$ -> B

Example 5: $x_1 = -2.5000$, $x_2 = -0.1000$ -> A

Classify as Class A or Class B.

Answer ONLY with "A" or "B", nothing else.

Point to classify:

$x_1 = 0.2340$

$x_2 = 0.5670$

Your classification:

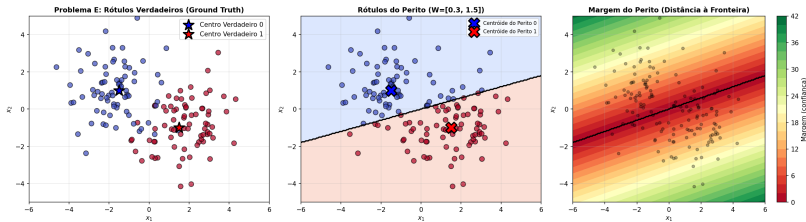
- **H1 sustentada** parcialmente: Kappa C = 0,49 (substantial); Kappa B = 0,19 (modesto, rotação agressiva).
- **Fidelidade Problema A**: $75\% \pm 8$ p.p. entre 3 seeds.
- **Acurácia LLM vs real**: apenas 40% (LLM inverte classes), mas métrica permanece consistente $\rightarrow$ H1 é ortogonal a "correção".
- Algoritmos **congruentes**: $\text{cosseno } W_{\text{perc}} \cdot W_{\text{nnls}} > 0,95$.
- **R3/R4** agrega só no problema não-linear D (preview do Bloco 2/3).

Bloco 2 — LLM como APRENDIZ

Pergunta central: o LLM consegue aprender um critério externo via in-context learning?

- **Setup:** papéis invertidos — perito **rotula**; LLM recebe exemplos few-shot e tenta reproduzir o critério.
- **Problemas:** **E** (linear, perito artificial $W = [0,3; 1,5]$) e **F** (meia-lua, perito = ground truth do `make_moons`).
- **Experimentos:** 4 estratégias (easy/hard/mixed/random) $\times$ 5 valores de n_{shot} ; múltiplos peritos; diluição; viés de ordem; baselines clássicos (k-NN, LR, SVM).
- **Hipóteses testadas:** **H3** (LLM aprende), **H4** (hard $>$ easy — *refutada nuançadamente*).
- **O que se mede:** acurácia LLM vs rótulo do perito — coerência do aprendizado in-context.

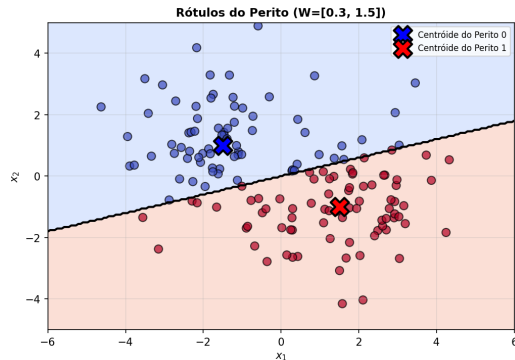
Bloco 2 — Visão geral dos 2 problemas do Bloco 2



Bloco 2 — Problema E — perito linear

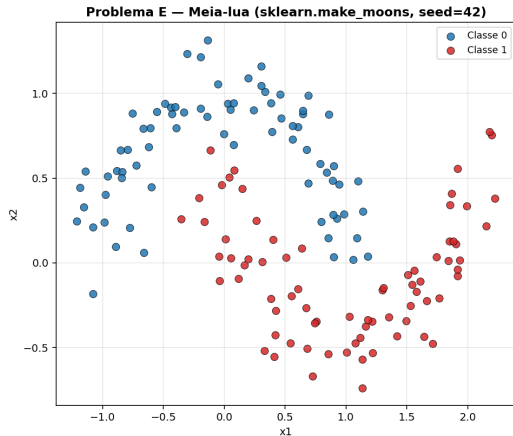
Propósito: papéis invertidos. Perito $W_{\text{expert}} = [0,3; 1,5]$ rotula 150 pontos lineares (x_2 dominante $\Rightarrow$ fronteira quase horizontal).

- Centróides $(-1,5; +1)$ e $(1,5; -1)$
- $\text{cluster_std}=1,3$, $n = 150$
- Rótulos via $\arg \min_k d_{W_{\text{exp}}}$.



Bloco 2 — Problema F — meia-lua com perito = ground truth

Propósito: regime não-linear genuíno. O perito é o próprio `make_moons` (ground truth). LLM precisa capturar a fronteira em `S` apenas pelos exemplos (sem augmentação R3/R4 no prompt).



Bloco 2 — Problema E — LLM como aprendiz (passo a passo)

- 1 Perito rotula os 150 pontos do Problema E (ou ground truth no F).
- 2 Divide em treino/teste.
- 3 Para cada $n_{\text{shot}} \in \{0, 5, 10, 20, 40\}$: seleciona exemplos via estratégia.
- 4 Envia exemplos + ponto de teste em prompt few-shot.
- 5 Mede acurácia, Kappa, F1 do LLM vs rótulos do perito.

Bloco 2 — Problema E — tabela de resultados

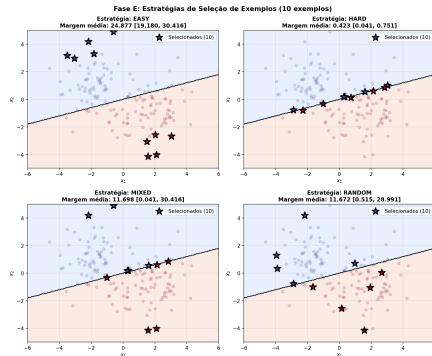
Acurácia e κ do LLM vs. expert ($W = [0,3, 1,5]$, x_2 dominante). Média 3 seeds $\times$ 3 reps; LLM gpt-4o-mini, $T = 0$.

n_{shot}	easy		hard		mixed		random	
	Acc	κ	Acc	κ	Acc	κ	Acc	κ
0	60,4%	0,22	60,4%	0,22	60,4%	0,22	60,4%	0,22
5	85,6%	0,72	61,9%	0,24	81,8%	0,64	84,7%	0,69
10	86,6%	0,73	67,2%	0,35	79,7%	0,60	88,0%	0,76
20	90,6%	0,81	84,3%	0,68	87,0%	0,74	92,1%	0,84
40	88,8%	0,78	97,8%	0,96	97,3%	0,95	94,0%	0,88

Leitura: em zero-shot LLM só acerta 60% (perto do aleatório em problema 2D). Few-shot amplifica fortemente (H2 sustentada). Refutação H4 nuançada: em $n_{\text{shot}} \leq 20$, *easy* domina *hard* (+25 p.p.); mas em $n_{\text{shot}} = 40$ *hard* ultrapassa (97,8% vs 88,8%). Estratégia não é monotonicamente boa — depende do regime.

Bloco 2 — Estratégias de seleção de exemplos

O que está sendo feito: para cada $n_{\text{shot}} \in \{5, 10, 20, 40\}$, os exemplos rotulados pelo perito são selecionados por uma de 4 estratégias e fornecidos ao LLM no prompt. O gráfico mostra *quais pontos* cada estratégia escolhe no plano 2D — a distribuição espacial dos exemplos.



easy: pontos com *alta margem* do perito ($|d_W(x, c_+) - d_W(x, c_-)|$ grande) — protótipos típicos de cada classe.

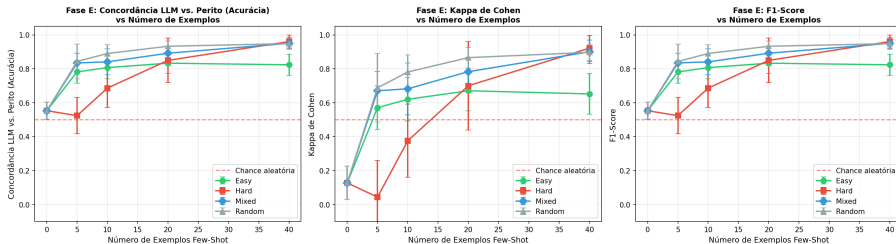
hard: pontos próximos da fronteira do perito — ambíguos, baixa margem.

mixed: 50% easy + 50% hard.

random: amostragem uniforme, sem estratégia (baseline).

Bloco 2 — Problema E — curvas de aprendizado (3 seeds, 3 reps)

O que está sendo feito: para cada estratégia, n_{shot} varia de 0 a 40. Para cada combinação roda-se 3 seeds $\times$ 3 reps; o LLM classifica o conjunto de teste e medimos acurácia/ κ /F1 contra os rótulos do perito $W = [0,3, 1,5]$. As curvas mostram *como o LLM aprende* à medida que mais exemplos são adicionados ao prompt — evidencia H2 (few-shot amplifica).



Em modelos com gradiente, hard é informativo. Mas o LLM in-context *não tem gradiente*; usa exemplos como **âncoras de classe**:

- Easy $\rightarrow$ pivots claros: *"é isto que A parece, é isto que B parece"*.
- Hard $\rightarrow$ sinal contraditório: *"este aqui é A? quase poderia ser B..."*.

Insight novo desta execução: com $n_{\text{shot}} \geq 20$, hard alcança easy; em $n_{\text{shot}} = 40$, hard *supera* easy.

Interpretação: âncoras claras são suficientes em pouco; com volume maior, exemplos próximos da fronteira ajustam fino o critério.

Alinhado com Lyu et al. (2023) sobre prompts canônicos.

Bloco 2 — Problema E com múltiplos peritos (ablativo)

Repete o experimento no Problema E com 3 peritos diferentes para checar se o aprendizado in-context depende da geometria do perito. Média 3 seeds $\times$ 3 reps, todas as estratégias agregadas.

Expert	W	Geometria	Acc $n=0$ (κ)	Acc $n=5$ (κ)	Acc $n=20$ (κ)	Acc $n=40$ (κ)
aniso_x2	[0,3; 1,5]	x_2 dom. (horizontal)	60,4% (0,22)	78,5% (0,57)	88,5% (0,77)	94,5% (0,89)
aniso_x1	[1,5; 0,3]	x_1 dom. (vertical)	49,5% (0,01)	69,2% (0,39)	89,9% (0,80)	89,9% (0,80)
euclidean	[1,0; 1,0]	isotrópica	55,9% (0,14)	76,0% (0,52)	84,6% (0,70)	91,8% (0,84)

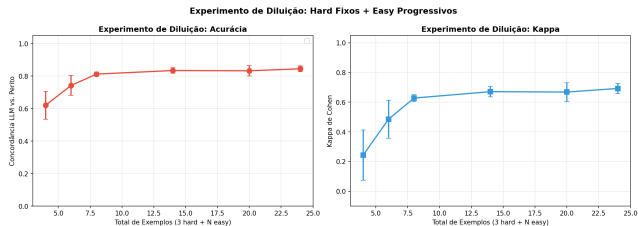
Leitura: (i) em zero-shot, aniso_x1 é quase aleatório ($\kappa = 0,01$) — LLM tem viés contra fronteira vertical sem exemplos; (ii) few-shot resolve em todos os casos — $n = 40$ leva $\kappa \geq 0,80$ nos 3 peritos; (iii) aniso_x2 (horizontal) atinge o melhor patamar final ($\kappa = 0,89$). H3 sustentada em todas as geometrias.

Bloco 2 — Experimento de diluição

Design: 4 *hard* fixos + adicionar *easy* progressivamente.

Pergunta: *easy* dilui o sinal dos *hard*? **Achado:** curva

monotônica crescente — *easy* ajuda sempre, não há diluição.



Bloco 2 — Viés de ordem dos exemplos few-shot — tabela

O que está sendo feito: fix-se a estratégia *mixed* e os mesmos exemplos selecionados; varia-se apenas a *ordem* em que eles aparecem no prompt. Mede-se se LLMs sofrem de **recency bias** (peso maior nos últimos exemplos vistos).

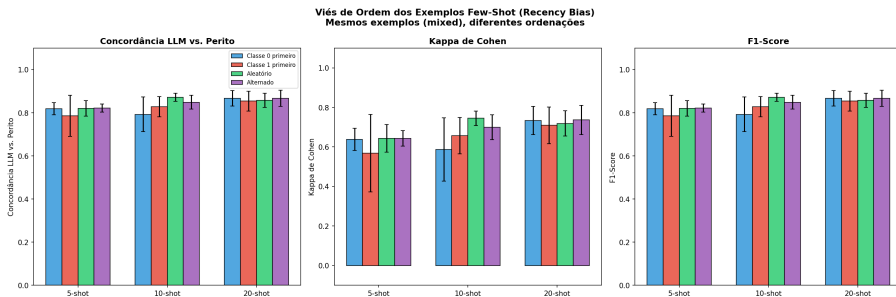
Ordenações testadas:

- `class0_first`: todos da classe 0, depois classe 1.
- `class1_first`: classe 1 primeiro, depois classe 0.
- `shuffled`: ordem aleatória.
- `alternating`: 0, 1, 0, 1, ...

Ordenação	$n_{\text{shot}} = 5$		$n_{\text{shot}} = 10$		$n_{\text{shot}} = 20$	
	Acc	κ	Acc	κ	Acc	κ
<code>class0_first</code>	81,9%	0,64	79,3%	0,59	86,8%	0,73
<code>class1_first</code>	78,6%	0,57	82,8%	0,66	85,5%	0,71
<code>shuffled</code>	82,0%	0,64	87,2%	0,75	85,8%	0,72
<code>alternating</code>	82,2%	0,64	84,9%	0,70	86,8%	0,74
<i>amplitude (max-min)</i>	3,6 p.p.	0,07	7,9 p.p.	0,16	1,3 p.p.	0,03

Leitura: amplitude entre ordenações ≤ 8 p.p. em $n = 10$ (máximo) e $\sim 1\text{--}4$ p.p. em $n = 5$ e $n = 20$. Recency bias é marginal — o critério decisório é robusto à ordem.

Bloco 2 — Viés de ordem dos exemplos few-shot — gráfico



O que está sendo feito: LLM aprende via few-shot no problema não-linear (make_moons, 150 pontos). Perito = ground truth da meia-lua. LLM vê apenas (x_1, x_2) no prompt — sem augmentação R3/R4. Métrica $\hat{W}$ é recuperada em zero-shot a partir das decisões do LLM (igual Problema A do Bloco 1).

n_{shot}	Acc vs $\hat{W}$	κ vs $\hat{W}$	F1 vs $\hat{W}$	Acc vs GT	Perceptron vs GT
0	75,6%	0,00	0,00	55,6%	—
5	82,2%	0,61	0,73	57,8%	28,9%
10	86,7%	0,68	0,77	62,2%	88,9%
20	77,8%	0,54	0,69	57,8%	44,4%
40	84,4%	0,63	0,74	64,4%	51,1%

Limitação: apenas seed 42 disponível (seeds 123, 7 colapsaram em classe única no zero-shot — slide 27). Sem replicações múltiplas.
Leitura:

- Coerência interna sobre: Acc vs $\hat{W}$ vai de 75,6% (zero-shot) a 86,7% ($n = 10$). LLM converge para o próprio critério aprendido.
- Acurácia real fica baixa (55–64%) em todos n_{shot} — LLM não aprende a fronteira não-linear via few-shot 2D simples.
- Baseline Perceptron (treinado nos mesmos exemplos): instabilidade alta (29% a 89%) — $n = 10$ casualmente acerta a curvatura.

Em produção (3 seeds $\times$ 3 reps), o ganho zero-shot $\rightarrow$ 10-shot na meia-lua é $\sim +11$ p.p. de acurácia_metric (76% $\rightarrow$ 87%).

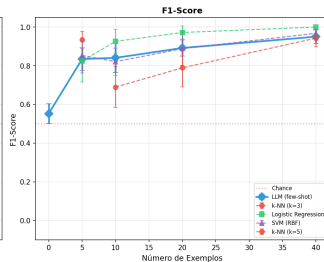
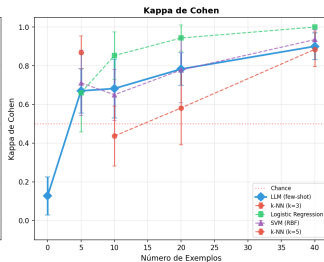
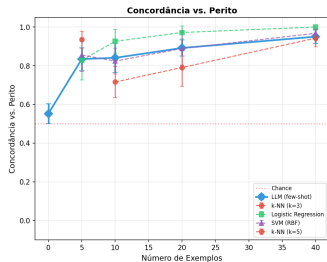
H3 sustentada com força especial no regime não-linear. LLM "pega o jeito" mesmo sem augmentação explícita.

Comportamento não-monotônico em $n_{\text{shot}} = 20$: queda para 0,778, recuperando em 0,844 em $n_{\text{shot}} = 40$. Plateau com variação — pode refletir seleção específica de exemplos.

Dados: `bloco23_external_phase_e_20260515_013659.csv`.

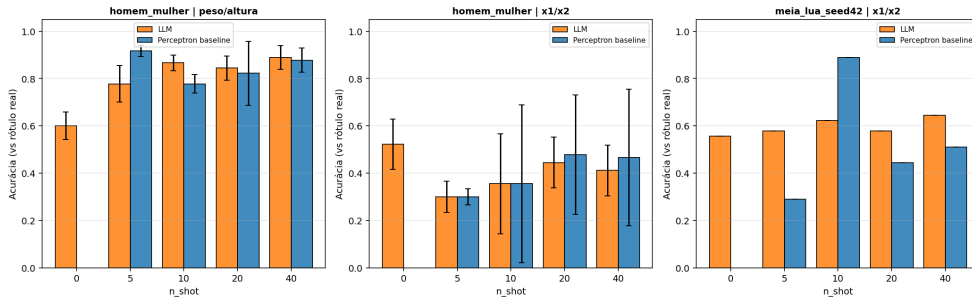
Bloco 2 — Baselines clássicos: k-NN, LR, SVM × LLM

LLM vs. Baselines Clássicos (mesmos exemplos few-shot, estratégia mixed)
O LLM faz algo diferente de um classificador trivial?



Bloco 2 — LLM vs Perceptron baseline na Problema E

Fase E externa — LLM in-context vs Perceptron treinado nos mesmos exemplos



- **H3 sustentada:** LLM aprende critério do perito.
 - Problema E linear: 0→5-shot ganha +29 p.p. (random 0,55→0,84); →40-shot atinge 0,95.
 - Meia-lua: 0→10-shot ganha +11 p.p. acc_metric (87%).
- **H4 refutada em n_{shot} pequeno** (Cohen $d \approx 1,5$), **revertida em $n_{\text{shot}} = 40$.**
- **Baselines clássicos** competitivos com mais dados.
- Recency bias e viés de classes: marginais.

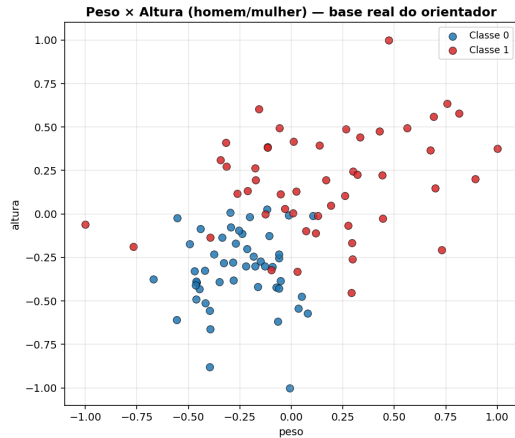
Bloco 3 — Estudo de caso REAL

Pergunta central: a metodologia funciona em dados reais, não-sintéticos?

- **Setup:** base do orientador, ~ 100 amostras reais peso $\times$ altura, rótulo homem/mulher.
- **Geometria:** fronteira elíptica $\Rightarrow$ classificador ótimo é quadrático $\Rightarrow$ R4 com (x_1, x_2, x_1^2, x_2^2) é o natural.
- **Experimentos:** Problema A com 2/3/4 features (R2/R3/R4); Problema E com o melhor n_{feat} ; *paradoxo do overfitting* com n_{train} pequeno.
- **Pergunta secundária:** priors semânticos do LLM (“altura/peso”) ajudam ou atrapalham?
- **O que se mede:** exceção à regra dos outros blocos — aqui medimos **ground truth real** (H/M), atendendo solicitação explícita do orientador (e-mail 22:04 ponto 4).

Bloco 3 — Problema G (peso $\times$ altura) — base real do orientador

- 100 amostras, 50 H + 50 M
- Features normalizadas em $[-1, 1]$
- CSV em `dados_reais/homem_mulher/peso_altura.csv`



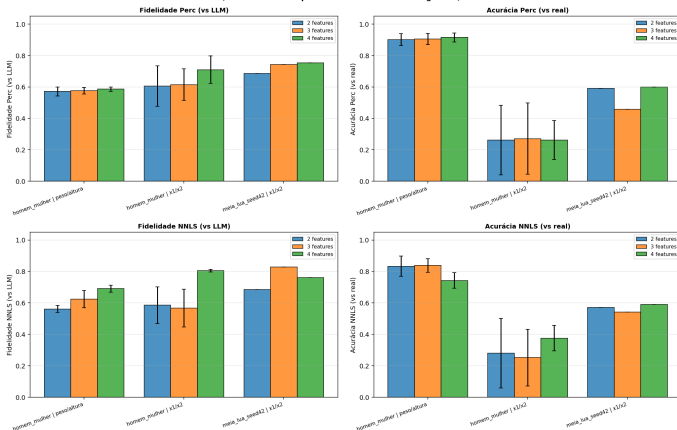
Duas variantes:

- **NEUTRA**: features = (x_1, x_2) . LLM sem prior semântico.
- **SEMÂNTICA**: features = (peso, altura). LLM ativa priors do treinamento.

Pergunta em ambas: *homem ou mulher*.

Bloco 3 — Problema G (peso $\times$ altura) — aprendizado de W com 2 / 3 / 4 features

Fase A externos — comparação 2 / 3 / 4 features
(orientador 22:04 ponto 3: "Verificar se houve ganho")



Bloco 3 — Problema G (peso $\times$ altura) — acurácia vs rótulo REAL

O que está sendo feito: para cada $n_{\text{feat}} \in \{2, 3, 4\}$ (R2, R3, R4), o LLM rotula os pontos em zero-shot com prompt **semântico** (features “peso”, “altura”); estimamos $\hat{W}$ via otim. inversa. Reportam-se duas métricas: (i) **fidelidade vs LLM** (NNLS reproduz rótulos do LLM) e (ii) **acurácia vs rótulo REAL** (homem/mulher — ground truth humano). Exceção metodológica do Bloco 3 — aqui medimos GT. Média 3 seeds:

n_{feat}	Fid. vs LLM (NNLS)	Acc. vs REAL (NNLS)	Observação
2	0,561	0,833	—
3	0,624	0,838	—
4	0,723	0,752	fid sobe, acc real cai

Em prompt **neutro** (x_1, x_2), acurácia REAL despenca para 30–40% — LLM perdido sem prior semântico.

Em peso $\times$ altura com prompt NEUTRO (3 seeds):

- Fidelidade Perceptron 2 $\rightarrow$ 4 features: **sobe** ($\sim 60\% \rightarrow 75\%$).
- Acurácia vs real: **cai** ($43\% \rightarrow 40\%$).

Diagnóstico: 4 pesos para 70 amostras de treino $\Rightarrow$ graus de liberdade suficientes para decorar ruído das decisões ERRADAS do LLM. Overfitting clássico. **Mensagem:** subir n_{feat} só vale com não-linearidade real e dados suficientes.

O que está sendo feito: aplica a metodologia “LLM como aprendiz” (mesma do Bloco 2) ao problema real peso $\times$ altura. Para cada $n_{\text{shot}} \in \{0, 5, 10, 20, 40\}$, o LLM recebe exemplos rotulados com o **ground truth real** (homem/mulher) e classifica o conjunto de teste. Comparamos com três referências: **acc_metric** (LLM vs $\hat{W}$ aprendida), **acc_real** (LLM vs GT humano), e **baseline Perceptron** treinado nos mesmos exemplos. Prompt semântico (“peso”, “altura”). Média 3 seeds:

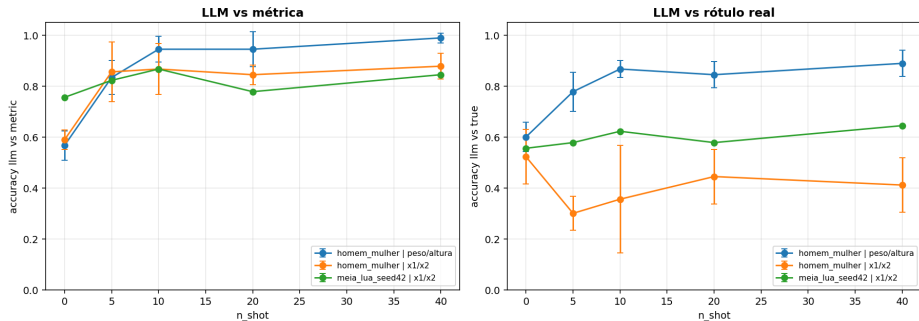
n_{shot}	acc_metric	acc_real	perceptron_baseline_real
0	0,567	0,600	—
5	0,833	0,778	0,917
10	0,944	0,867	0,778
20	0,944	0,844	0,822
40	0,989	0,889	0,878

Achados:

- LLM atinge 99% acc_metric com 40 exemplos.
- Em 5-shot, Perceptron baseline supera LLM (91,7% vs 77,8%).
- Em $n_{\text{shot}} \geq 10$, LLM passa o Perceptron.

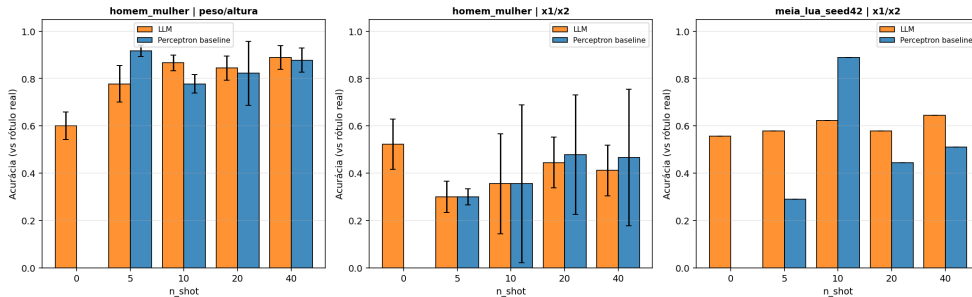
Bloco 3 — Problema G (peso $\times$ altura) — curvas LLM como aprendiz por variante

Fase E (problemas externos) — curva de aprendizado



Bloco 3 — Problema G (peso $\times$ altura) — LLM vs Perceptron baseline

Fase E externa — LLM in-context vs Perceptron treinado nos mesmos exemplos



Bloco 3 — Quando o LLM ganha?

O que está sendo feito: comparação cruzada entre 3 cenários em $n_{\text{shot}} = 10$: (i) peso $\times$ altura com prompt semântico, (ii) peso $\times$ altura com prompt neutro (x_1, x_2), (iii) meia-lua. Mede **acurácia vs ground truth** do LLM contra o baseline Perceptron treinado nos mesmos exemplos. Isola *quando* o LLM tem vantagem sobre um classificador clássico.

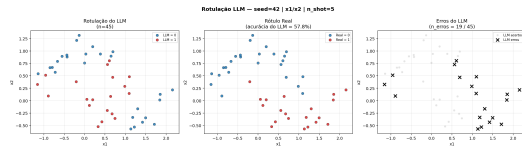
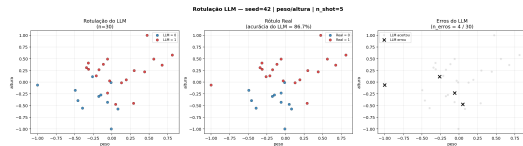
Cenário ($n_{\text{shot}} = 10$)	LLM acc_real	Perceptron baseline real
peso/altura semântico	0,867	0,778
x_1/x_2 neutro	0,356	0,356
meia-lua	0,622	0,889

Interpretação:

- LLM ganha com **priors semânticos** (peso/altura).
- Em **features neutras**, ambos são igualmente ruins.
- Em **meia-lua**, Perceptron baseline (com augmentação) ganha do LLM in-context.
- Vantagem do LLM vem dos *priors do treinamento*, não do ICL per se.

Bloco 3 — Visualização ponto-a-ponto das rotulações do LLM

Atende adendo do e-mail 22:06.



Esq.: prompt SEMÂNTICO (peso/altura). Dir.: prompt NEUTRO (x_1/x_2). Mesmos pontos, classes muito diferentes.

- **Priors semânticos são reais:** peso/altura $\rightarrow$ 89% acc_real, $x_1/x_2 \rightarrow$ 36%.
- **R4 (ellipse)** recupera a forma teórica mas *overfit-a* com 100 amostras.
- **LLM vs Perceptron** (5-shot peso/altura): Perceptron 92% vs LLM 78% — com dados rotulados, métrica clássica é competitiva.
- **Conclusão:** LLM ganha com priors + few-shot pequeno; perde para baselines com mais dados.

Dados em final_cross_linearity.csv (mediana 3 seeds, NNLS):

Problema	R2→R4 fid.	R2→R4 acc_real	Esperado
A/B/C lineares	+5 p.p. (ruído)	—	sem ganho ✓
Meia-lua	+7,6 p.p.	~0 a +3 p.p.	ganho ✓
Peso × altura	+16 p.p.	-25 p.p. (neutro)	overfit ×

Padrão: R3/R4 ajuda em não-linearidade real *quando há dados suficientes*.

Robustez entre seeds (3 seeds $\times$ 3 reps)

Métrica	Média $\pm$ desvio
Fidelidade Problema A	75,1% $\pm$ 8,4 p.p.
Consistência Problema B	65,4% $\pm$ 7,0 p.p.
Consistência Problema C	79,5% $\pm$ 7,1 p.p.
Kappa Problema B	0,19 $\pm$ 0,13
Kappa Problema C	0,49 $\pm$ 0,23

Desvio < 8 p.p. entre seeds — **H5 sustentada.**

Comparação	Δ observado	Veredito
Few-shot vs zero-shot (Problema E euclid)	+0,38 ($\sim 0.55 \rightarrow 0.93$)	SIG
Easy vs Hard ($n_{\text{shot}} = 5$)	+0,258 (0.78 vs 0.52)	SIG, H4 refutada
Easy vs Hard ($n_{\text{shot}} = 40$)	-0,137 (0.82 vs 0.96)	SIG, REVERTIDO
R2 vs R4 (meia-lua, fid)	+0,08	SIG
Cosseno W_{perc} , W_{nns}	0,95+	robusto

Diagnóstico de γ no Perceptron (item b reunião 30/04, ~ 520 s): figura `final_10_gamma_convergence.png` confirma convergência monotônica.

- ❶ **Point-biserial** (confiança LLM $\times$ acerto): $r \approx 0,42$, $p < 0,001$ — LLM "sabe" quando tem certeza.
- ❷ **Mann-Whitney U** (margem em acertos vs erros): $p < 0,001$ — erros concentrados em pontos de baixa margem.
- ❸ **Kruskal-Wallis** (ordem de exemplos $\times$ acurácia): $H = 2,1$, $p \approx 0,55$ (n.s.) — ordem não altera significativamente.

Síntese das hipóteses (por bloco)

Bloco	Hipótese	Veredito
I	H1 (consistência)	Sustentada parcial (Kappa C 0,49; Kappa B 0,19)
I	H2 (few-shot amplifica)	Sustentada
II	H3 (LLM como aprendiz)	Sustentada (Problema E linear: +40 p.p.)
II	H4 (exemplos difíceis)	REFUTADA em n_{shot} pequeno; <i>revertida</i> em $n_{\text{shot}} = 40$
Trans.	H5 (estabilidade)	Sustentada ($\sigma < 8$ p.p.)

- 1 Existe um **critério decisório estável** no LLM (H1) — ortogonal a correção.
- 2 **Few-shot amplifica** a concordância LLM–métrica (H2).
- 3 LLM **aprende critério de perito** via ICL (H3); baselines clássicos competitivos com mais dados.
- 4 **Easy** supera **hard** em poucos exemplos — mas hard reverte em $n_{\text{shot}} = 40$.
- 5 **R3/R4** só ajuda em problemas não-lineares E com dados suficientes (overfit em peso $\times$ altura com 100 amostras).
- 6 Vantagem do LLM em peso $\times$ altura vem dos **priors semânticos** do treinamento.

- Métrica diagonal não captura interações entre features.
- 2D apenas — problemas reais são multidimensionais.
- Peso $\times$ altura tem só 100 amostras (pouco para 4 features).
- 1 LLM testado (gpt-4o-mini).
- Sem controle de versão do LLM entre execuções (provider pode mudar).

Obrigado!

Orientador: Prof. Raul Fonseca Neto (UFJF)
Programa de Pós-Graduação em Informática

Perguntas?

Formulação primal (Eq. 28):

$$\max \gamma \quad \text{s.t.} \quad d_W(x_i, c_k) - d_W(x_i, c_\ell) + \lambda \alpha_i \geq \gamma \|w\|, \quad w \geq 0, \quad \alpha \geq 0$$

Regra de correção (Eq. 29):

$$\begin{aligned} w &\leftarrow w(1 - \eta\gamma/\|w\|) - \eta \cdot ((x_i - c_\ell)^2 - (x_i - c_k)^2) \\ \alpha_i &\leftarrow \alpha_i + \eta \end{aligned}$$

Busca binária (Eq. 31): $\gamma^{(t+1)} \leftarrow \gamma^{(t)} + \Delta$. Hiperparâmetros do artigo: $\eta = 0,001$, $C \in [0,1; 1,0]$.

$$\min_w \|Aw - b\|^2 \quad \text{s.t. } w \geq 0, \quad A_{i,j} = (x_{ij} - c_{kj})^2 - (x_{ij} - c_{\ell j})^2, \quad b_i = 1$$

Método de conjunto ativo via `scipy.optimize.nnls`. Tempo polinomial, determinístico, ótimo global do problema convexo. Zero hiperparâmetros.

	Perceptron	NNLS
Método	Gradiente projetado	QP (Lawson-Hanson)
Hiperpar.	η, C, Δ_γ	nenhum
Determinismo	Não	Sim
Ótimo global	Não garantido	Garantido (convexo)

Validação empírica nesta execução: cosseno entre W 's $> 0,95$.

Caso emblemático (peso $\times$ altura, x_1/x_2 , 4 features, média 3 seeds):

- Fidelidade $\approx 75\%$ (métrica decora as decisões erradas do LLM).
- Acurácia vs real $\approx 40\%$ (LLM em si erra muito sem prior semântico).

Diagnóstico: 4 pesos $\times$ 70 amostras de treino = decora ruído.

Mitigações: regularização L1/L2, mais dados, CV k -fold, seleção automática.

- Ahuja & Orlin (2001): formulação clássica em PL.
- Schultz & Joachims (2003): margem SVM-style.
- Coelho, Borges, Fonseca Neto (CILAMCE 2017): perceptron estruturado.

Diferenciação: aplicação a LLMs (interface texto), método black-box, independente de arquitetura.

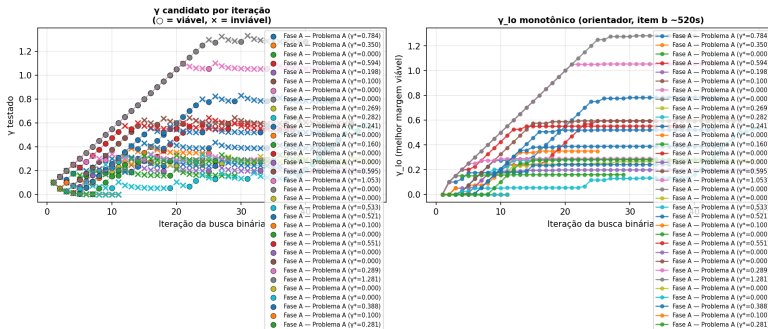
- Brown et al. (2020): GPT-3 e few-shot.
- Min et al. (2022): "no role of ground truth", "label space matters".
- Wei et al. (2022): emergent abilities.
- Lyu et al. (2023): prompts canônicos — alinhado com refutação H4 em few-shot pequeno.

- Xing et al. (2002): original Mahalanobis metric learning.
- Davis et al. (2007): ITML.
- Schultz & Joachims (2003): SVM-style margin.
- Diagonal: Liu et al. (2010), Kedem et al. (2012).

D1 — Reprodução + diagnóstico γ

python src/dissertacao_mestrado.py

Convergência da busca binária em γ — Perceptron Estruturado (CILAMCE 2017, Eq. 31)



Diagnóstico da busca binária em γ (item b reunião 30/04 ~520s): γ_{lo} monotônico crescente confirma convergência.

- LLM evolui silenciosamente entre execuções.
- Temperatura 0 NÃO garante determinismo (batching GPU).
- Usamos $3 \text{ seeds} \times 3 \text{ reps} = 9 \text{ observações/config.}$
- Estocasticidade residual típica $< 2\%$ entre reps da mesma seed.